

AdaVLA: Adaptive Step Flow Matching for Training-free Acceleration of Vision-Language-Action Models

Sunghwan Han¹, Youngtae Han¹, and Youngmin Yi^{1†}

Abstract—Vision-Language-Action (VLA) models, built upon Vision-Language Models (VLMs), have significantly enhanced robotic capabilities by leveraging internet-scale knowledge and multimodal reasoning. However, the intensive computational overhead of VLAs constrains on-device deployment, hindering real-time responses to environmental changes.

While various acceleration techniques have been proposed, they often rely on fine-tuning or access to training datasets, which are frequently unavailable due to privacy and proprietary concerns. Moreover, although flow-matching-based VLAs have emerged as efficient alternatives to standard diffusion models, current acceleration efforts largely target VLM inference costs, failing to address the iterative ODE solving process inherent in flow matching inference. To address these limitations, we propose *AdaVLA*, an online, training-free adaptive framework for fast yet accurate flow-matching-based Vision-Language-Action models. We introduce a novel metric derived from the flow matching trajectory curvature to quantify action generation confidence during inference. This metric enables the dynamic reduction of inference steps and the adaptive adjustment of MLP pruning ratios through an efficiently computed importance evaluation, requiring no access to training data. Experimental results on the LIBERO benchmark using a Jetson AGX Orin device demonstrate that our method achieves $1.87\times$ and $2.24\times$ speedups for $\pi_{0.5}$ and X-VLA, respectively, with negligible degradation in success rates. Furthermore, we validate the robustness of our approach on real-world robotic tasks using SmoVLA.

I. INTRODUCTION

Recent advancements in integrating visual and language models have led to the development of Vision-Language Models (VLMs), marking significant progress in multimodal reasoning and generalization. This progress has extended to robotic action planning, evolving VLMs from generating textual responses to predicting direct control actions, resulting in Vision-Language-Action (VLA) models [1], [2], [3]. These models demonstrate superior robustness and generalization across diverse environments compared to traditional reinforcement learning policies.

However, VLAs inherit the intensive computational costs of the underlying VLM. This results in substantial inference latency between actions, leading to unnatural robotic motion. While several acceleration methods—including lightweight design (TinyVLA [4]), early-exiting (DeeR-VLA [5]), and token pruning with layer-skipping (EfficientVLA [6])—have been proposed, they typically require access to training data for fine-tuning or calibration. In many practical settings,

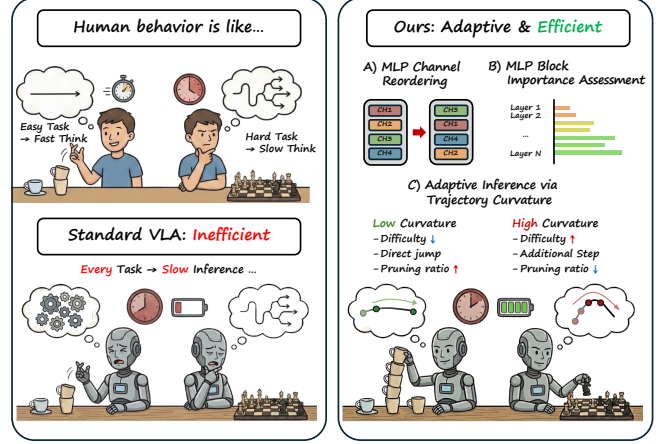


Fig. 1: Concept of *AdaVLA*, our proposed online adaptive acceleration framework.

however, such access is restricted due to proprietary concerns, and the sheer volume of these datasets often prohibits processing on edge devices.

Furthermore, current acceleration strategies [7], [8] primarily focus on reducing the computational load of the VLM backbone. However, the emergence of flow-matching [9]-based VLAs, such as the π series [10], [11], SmoVLA [12], and X-VLA [13], has changed this landscape. In these architectures, the VLM serves as an encoder that passes context via the key-value (KV) cache to the flow-matching-based Action Expert through cross-attention. As illustrated in our subsequent latency analysis (Sec. III-C), the iterative process of generating actions from noise within the Action Expert now accounts for a larger portion of the total latency than the VLM backbone. Consequently, optimizing only the VLM is insufficient for accelerating state-of-the-art flow matching VLAs.

To address these challenges, we propose *AdaVLA*, a training-free, online adaptation framework that simultaneously reduces computational costs for both the VLM and the Action Expert during runtime. Drawing inspiration from human cognition—where simple tasks elicit quick, intuitive responses while complex ones demand deeper deliberation—we enable flow-matching-based VLAs to autonomously adapt their computational effort based on the perceived difficulty of the task at hand, as illustrated in Fig. 1.

Our methodology consists of three key components that serve as an end-to-end pipeline for acceleration:

- (1) **Curvature-based Adaptive Inference:** We leverage the flow-matching trajectory curvature to quantify action generation confidence during inference and dynamically

¹All authors are with the Dept. of Artificial Intelligence, Sogang University, Seoul, 04107, Republic of Korea. {sunghwan06, young0tete, ymyi}@sogang.ac.kr

[†] Corresponding author.

adapt both the inference step size and the MLP pruning ratio during the ODE solving process (Sec. IV-A).

- (2) **MLP Block Importance Assessment:** Since naively pruning MLP channels across all layers in a uniform manner can degrade the success rate, we introduce an efficient, SVD-free assessment to evaluate the importance of an entire MLP block in each layer. This allows the framework to efficiently preserve representational diversity while simultaneously reducing computational costs (Sec. IV-B).
- (3) **MLP Channel Reordering:** We reorder intermediate channels within an MLP block in descending order of an importance metric to account for dynamic changes in importance. This is selectively triggered during the initial forward pass or upon detecting a significant context shift, enabling efficient yet effective structured pruning without training data access (Sec. IV-C).

Through evaluations on $\pi_{0.5}$ [11] and X-VLA [13] using a Jetson AGX Orin, we demonstrate $1.87\times$ to $2.24\times$ latency reductions on the LIBERO benchmark [14] with minimal impact on the success rate. We also demonstrate the effectiveness of AdaVLA in real-world robotic tasks on the SO-ARM101 [15] using SmolVLA [12].

II. RELATED WORK

A. Vision-Language-Action (VLA) Models

VLA models have emerged as a new paradigm for general-purpose robot learning. Early developments such as RT-1 [16] utilized Transformers to output discrete actions, while RT-2 [2] demonstrated that co-fine-tuning VLMs on large-scale datasets enables the transfer of semantic reasoning to robotic control tasks. Moving beyond autoregressive policies like OpenVLA [1], continuous generative approaches have been introduced for modeling complex action distributions. Diffusion Policy [17] pioneered the application of diffusion models to visuomotor control, leading to integrated VLA architectures such as CogACT [18] and DiffusionVLA [19]. Recently, flow matching [9] models, including π_0 [10], $\pi_{0.5}$ [11], SmolVLA [12], and X-VLA [13], have gained prominence. These models leverage continuous normalizing flows to achieve high-quality generation with significantly fewer steps, making them effective for real-time control.

B. Efficient Vision-Language-Action Models

Despite their capabilities, the substantial computational costs of VLAs hinder their deployment on resource-constrained edge devices. To address this challenge, various acceleration techniques have been proposed. RoboMamba [20] introduced State Space Models (SSMs) [21] for robotic reasoning tasks, achieving linear complexity with respect to sequence length. DeeR-VLA [5] proposed a dynamic early-exiting mechanism, allowing the model to terminate inference early for simple tasks to reduce computation. OpenVLA-OFT [22] enables efficient parallel decoding for large-scale backbones. Other efforts mitigate Transformer

redundancy; for instance, EfficientVLA [6] and MoLe-VLA [23] utilize token/layer pruning and dynamic layer-skipping to enhance efficiency. However, these approaches rely on retraining or calibration (e.g., layer importance assessment), both of which typically require training data. This dependency limits their applicability in data-limited and privacy-sensitive scenarios, which our work addresses.

III. PRELIMINARY

A. Formulation and VLM Encoding

A Vision-Language-Action (VLA) model learns a policy $p(A_t|o_t)$ that maps an observation o_t at time t to an action chunk $A_t \in \mathbb{R}^{H \times D}$, where H denotes the action horizon and D represents the action dimension. The observation o_t is a multimodal input consisting of images $I_t^1, \dots, I_t^n$, a language instruction ℓ_t , and the current robot state q_t :

$$o_t = [I_t^1, \dots, I_t^n, \ell_t, q_t] \quad (1)$$

The VLM encodes this observation o_t through a single forward pass. During this process, the resulting Key-Value (KV) cache is shared with the Action Expert to perform cross-attention, allowing the model to utilize multimodal information while generating actions from noise.

B. Action Expert & Flow Matching

The Action Expert generates the action A_t using a conditional flow matching [9] framework that learns a vector field along an interpolation path from noise to data. A linear path between the Gaussian noise ϵ and the clean action A_t , parameterized by the flow-matching time variable $\tau \in [0, 1]$, is defined as follows:

$$A_t^\tau = \tau A_t + (1 - \tau)\epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (2)$$

During training, the model learns a vector field that induces a trajectory from the noise to the clean action. The target vector field $\mathbf{u}$ along this path is given by the derivative of A_t^τ with respect to τ :

$$\mathbf{u}(A_t^\tau|A_t) = \frac{d}{d\tau} A_t^\tau = A_t - \epsilon \quad (3)$$

Since the target vector field $\mathbf{u}$ is τ -independent, the induced target trajectory is linear. This formulation enables efficient ODE integration and can reduce the number of solver steps in practice. The model $\mathbf{v}_\theta$ is trained to approximate the target vector field $\mathbf{u}$ by minimizing the squared error between them, with τ uniformly sampled:

$$\mathcal{L}(\theta) = \mathbb{E}_{\tau, \epsilon, (o_t, A_t)} \left[\|\mathbf{v}_\theta(\tau, A_t^\tau, o_t) - \mathbf{u}(A_t^\tau|A_t)\|^2 \right] \quad (4)$$

At inference, starting from the noise state ($\tau = 0$), the model solves the Ordinary Differential Equation (ODE) toward the data state ($\tau = 1$) conditioned on the observation o_t . Specifically, an Euler solver is employed to transform an initial noise sample into the action sequence.

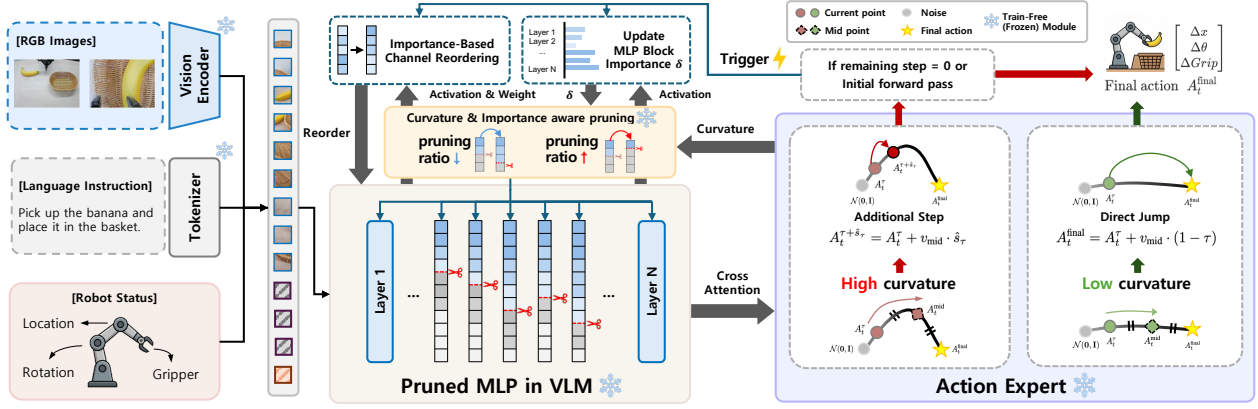


Fig. 2: Overall architecture of AdaVLA, our proposed online adaptive acceleration framework.

TABLE I: Inference latency of $\pi_{0.5}$ on Jetson AGX Orin

Module	Component	Latency (ms)
VLM Backbone (18 Layers)	Processing Input	96.25
	VLM Forward Pass	290.28
	└ Self-Attention	└ 3.05
	└ MLP Block	└ 12.47
Action Expert (10 steps)	Flow Matching Process	557.78
	└ Single Step	└ 53.75
Total Inference	End-to-End	953.62

C. Latency Analysis

To analyze the performance bottlenecks on edge devices, we measured the average inference latency of $\pi_{0.5}$ [11] on a Jetson AGX Orin while performing the LIBERO benchmark [14]. The latency profile for action generation is summarized in Table I, and the key findings from the analysis are as follows:

- **VLM Bottleneck (MLP vs. Attention):** The VLM backbone accounts for approximately 40% (386.53 ms) of the total inference latency. Within its 18 layers, the MLP blocks (12.47 ms per layer) are significantly more computationally intensive than the self-attention blocks (3.05 ms per layer), which aligns with recent findings in LLM optimization [24], [25]. Although the Action Expert consumes a larger proportion of the latency, targeting MLP blocks within the VLM provides an essential control knob to optimally balance the latency–success rate trade-off.
- **Action Expert Bottleneck (Iterative ODE Solving):** The flow-matching process constitutes the primary bottleneck, representing approximately 58% (557.78 ms) of the total inference time. While individual steps are relatively fast (53.75 ms), the cumulative latency of 10 iterations is substantial. This highlights that adjusting the number of iterations based on trajectory curvature is essential for improving overall efficiency.

IV. METHOD

Fig. 2 illustrates the overall workflow of the proposed framework, AdaVLA. Our methodology comprises three primary components: (i) **Adaptive Inference** (Sec. IV-A), which reduces redundant ODE solver steps during action

generation within the Action Expert; (ii) **MLP Block Importance Assessment** (Sec. IV-B), a training-free and efficient scheme to modulate VLM backbone pruning ratios by leveraging Participation Ratio-based block importance; and (iii) **MLP Reordering** (Sec. IV-C), which facilitates structured pruning by accounting for channel-wise importance.

A. Adaptive Inference via Trajectory Curvature

Generating actions via flow matching is a primary bottleneck due to its iterative nature. We propose an online adaptation framework that dynamically adjusts flow matching step sizes and pruning ratios based on perceived difficulty.

As shown in Sec. III-B, the flow matching model learns a vector field that transports samples from the noise ϵ toward the action A_t . Unlike high-dimensional image generation, where straight trajectories are rarely guaranteed due to immense dimensionality, action generation operates in a significantly more compact space. Given that these action dimensions are substantially smaller than those of high-dimensional image spaces, we hypothesize that the induced sample trajectory would remain nearly linear unless the model encounters an uncertain state. Consequently, the straightness of trajectories can serve as a reliable proxy for model confidence. To quantify trajectory deviation, we define the relative curvature C between estimated vectors v_{τ_A} and v_{τ_B} ($\tau_A < \tau_B$), using the magnitude of v_{τ_B} as a normalization factor, where ε is a small constant for numerical stability:

$$C = \frac{\|v_{\tau_A} - v_{\tau_B}\|_2}{\|v_{\tau_B}\|_2 + \varepsilon} \quad (5)$$

Unlike standard flow matching’s Euler steps, AdaVLA draws inspiration from the second-order Runge-Kutta (RK2) method [26], [27] not only for enhanced trajectory estimation but also as an indicator of trajectory straightness. Although RK2 doubles the number of function evaluations (NFE = 2) per iteration, AdaVLA strategically bypasses redundant computations when the trajectory is nearly linear. At time τ , AdaVLA computes $v_\tau := \mathbf{v}_\theta(\tau, A_t^\tau, o_t)$, sets $\tau_{\text{mid}} = \tau + \frac{1-\tau}{2}$ and $A_t^{\text{mid}} = A_t^\tau + v_\tau \cdot \frac{1-\tau}{2}$, and then evaluates $v_{\text{mid}} := \mathbf{v}_\theta(\tau_{\text{mid}}, A_t^{\text{mid}}, o_t)$. Then, the curvature C , computed from v_τ and v_{mid} , is compared against a predefined threshold C_{th} , based on which the inference process branches dynamically.

Low Curvature ($C < C_{\text{th}}$): The trajectory is interpreted as nearly linear. Relying on v_{mid} to extrapolate the trajectory, the framework bypasses all remaining iterations to execute a *direct jump* to the final state ($\tau \approx 1$):

$$A_t^{\text{final}} = A_t^\tau + v_{\text{mid}} \cdot (1 - \tau) \quad (6)$$

Simultaneously, we aggressively increase the l -th layer's base pruning ratio p_l , modulated by the layer importance δ_l , while ensuring that it remains within a valid range:

$$p'_l = \min(1.0, p_l + \exp(1 - \delta_l) \cdot (C_{\text{th}} - C)) \quad (7)$$

where δ_l and p_l are detailed in Sec. IV-B.

High Curvature ($C \geq C_{\text{th}}$): The increased trajectory complexity necessitates a more conservative yet adaptive progression. Leveraging the pre-computed midpoint vector v_{mid} , we advance the state using a bounded step size $\hat{s}_\tau$. Since the RK2 method requires two function evaluations (NFE = 2) per iteration, we aim to utilize a step size of $2s$ to maintain computational efficiency comparable to standard solvers. However, to prevent success rate degradation in regions of high complexity, we modulate this target step size by C_{th}/C , allowing the model to dynamically scale down its progression based on the measured trajectory deviation. To ensure stability, we bound $\hat{s}_\tau$ to the range $[s, (1 - \tau)/2]$:

$$\hat{s}_\tau = \min\left(\frac{1 - \tau}{2}, \max\left(s, \frac{C_{\text{th}}}{C} \cdot 2s\right)\right) \quad (8)$$

We use v_{mid} for updates because, compared to the current-time evaluation v_τ , it is empirically more reliable (lower-variance) and yields more stable updates under the same compute budget. The next state is then updated as follows:

$$A_t^{\tau + \hat{s}_\tau} = A_t^\tau + v_{\text{mid}} \cdot \hat{s}_\tau \quad (9)$$

To restore reasoning capacity, the pruning ratio is decreased based on the layer importance δ_l :

$$p'_l = \max(0, p_l - \exp(\delta_l) \cdot (C - C_{\text{th}})) \quad (10)$$

If the maximum allowed steps are exhausted, *AdaVLA* interprets the situation as a significant context shift, triggering the recalculation of MLP block importance (Sec. IV-B) and MLP channel reordering (Sec. IV-C).

By consolidating these adaptive functionalities, *AdaVLA* encapsulates the latency–success rate trade-off into a single design parameter, the curvature threshold C_{th} . This eliminates the need for manual, multi-dimensional tuning of layer-wise pruning ratios and flow matching hyperparameters. As shown in the ablation study (Sec. V-C), there is a gradual trade-off between success rate and latency as C_{th} increases, allowing for scenario-specific optimization.

B. Training-free & Efficient MLP Importance Assessment

Applying a uniform pruning ratio to the MLP blocks across all layers is suboptimal, as individual layers exhibit varying degrees of parameter sensitivity and information density. Furthermore, computing gradient-based layer importance is often prohibitively expensive for edge devices

due to their limited computational budgets and data privacy constraints.

To address this, *AdaVLA* evaluates layer importance using only forward passes without access to training data, employing a metric inspired by the Effective Rank [28] of MLP activations. Our approach is grounded in the findings of RankMe [29], which demonstrate that the Effective Rank of neural representations is a robust indicator of downstream performance. This suggests that it can serve as a reliable proxy for a layer's expressivity; a higher rank implies that the representations are more diverse and capture essential features, whereas a lower rank indicates high redundancy. Specifically, for an activation matrix $A \in \mathbb{R}^{N \times M}$, the Effective Rank is defined using the Shannon entropy (H_1) of its singular value distribution:

$$\begin{aligned} \text{Effective Rank}(A) &= \exp(H_1(P)) \\ &= \exp\left(-\sum_{k=1}^{\min(N,M)} p_k \ln p_k\right) \end{aligned} \quad (11)$$

where p_k represents the normalized k -th singular value σ_k :

$$p_k = \frac{\sigma_k(A)}{\sum_{m=1}^{\min(N,M)} \sigma_m(A)} \quad (12)$$

Here, $\sigma_k(A)$ denotes the k -th singular value of A . This formulation accurately captures the “spread” of the singular value spectrum. By preserving high-rank blocks for representational diversity and aggressively pruning redundant low-rank ones, we can optimize the latency–success rate trade-off.

However, computing exact singular values via Singular Value Decomposition (SVD) introduces prohibitive latency on edge devices due to iterative numerical operations and GPU-CPU synchronization overhead. To bypass SVD, *AdaVLA* efficiently estimates the representational spread using the Participation Ratio, which is a computationally tractable variant of Effective Rank. Specifically, we leverage the symmetric Gram matrix $G = A^T A$, whose eigenvalues λ_m directly correspond to the squared singular values of the original activation matrix A .

By defining the normalized eigenvalue distribution of the Gram matrix G as $\tilde{p}_m = \lambda_m / \sum_n \lambda_n$ and replacing the Shannon entropy (H_1) with the Collision entropy (H_2), we derive the Participation Ratio (PR):

$$\begin{aligned} \text{PR} &= \exp(H_2(\tilde{P})) \\ &= \exp(-\ln(\sum_m \tilde{p}_m^2)) \\ &= \frac{1}{\sum_m \tilde{p}_m^2} = \frac{(\sum_n \lambda_n)^2}{\sum_m \lambda_m^2} \end{aligned} \quad (13)$$

Since the sum of eigenvalues equals the trace ($\sum_n \lambda_n = \text{tr}(G)$) and the sum of squared eigenvalues for a symmetric matrix G equals the squared Frobenius norm ($\sum_m \lambda_m^2 = \|G\|_F^2$), the Participation Ratio (PR) of the l -th layer can be computed efficiently without any decomposition:

$$\text{PR}_l = \frac{(\sum_n \lambda_n)^2}{\sum_m \lambda_m^2} = \frac{\{\text{tr}(G_l)\}^2}{\|G_l\|_F^2} \quad (14)$$

TABLE II: Comparison of *AdaVLA* with $\pi_{0.5}$ and X-VLA on the LIBERO Benchmark.

Base Model	Params	Method	Success Rate (%)					Latency (ms)		Energy (J)
			Spatial	Object	Goal	Long	Average	A10	Orin	Orin
$\pi_{0.5}$ [11]	3.3B	Baseline	99.40	98.20	98.80	98.20	98.65	289.68	958.54	19.83
		+ SparseVLM [7]	<u>96.60</u>	97.00	97.60	94.20	96.35	283.68	945.09	<u>17.35</u>
		+ FastV [8]	66.20	76.80	77.20	48.00	67.05	<u>265.11</u>	<u>863.84</u>	17.38
		+ Ours	99.40	100.00	99.80	98.60	99.45	158.87	513.35	13.17
X-VLA [13]	0.9B	Baseline	99.60	100.00	98.00	96.40	98.50	151.32	387.52	10.73
		+ SparseVLM [7]	99.20	82.00	97.00	90.60	92.20	<u>147.74</u>	389.60	<u>10.20</u>
		+ FastV [8]	98.40	99.60	98.00	81.20	94.30	148.00	<u>385.21</u>	10.31
		+ Ours	<u>99.40</u>	100.00	98.80	97.40	98.90	64.90	173.09	3.51

where G_l denotes the Gram matrix from the l -th layer. We normalize these Participation Ratios to obtain a relative importance score $\delta_l \in [0, 1]$ for the l -th layer:

$$\delta_l = \frac{\text{PR}_l - \min_j(\text{PR}_j)}{\max_j(\text{PR}_j) - \min_j(\text{PR}_j)} \quad (15)$$

Based on δ_l , highly important MLP blocks ($\delta_l \approx 1$) are pruned conservatively to preserve capacity, while less important ones ($\delta_l \approx 0$) are pruned aggressively.

C. MLP Channel Reordering

Table I shows that VLM computational bottlenecks stem primarily from the MLP blocks. Given that these bottlenecks arise from the massive intermediate channels, we adopt structured pruning, which is more efficient and implementation-friendly than unstructured alternatives. Since access to training data is often restricted after deployment, offline calibration for pruning becomes infeasible. To address this, *AdaVLA* dynamically reorders intermediate channels based on their channel importance metrics during inference, without requiring any calibration data. Flow-matching-based VLAs typically employ either Gated MLP [10], [11], [12] or 2-Layer MLP [13] with an activation function ϕ :

$$\text{Gated MLP}(x) = \text{Down}(\phi(\text{Gate}(x)) \odot \text{Up}(x)) \quad (16)$$

$$\text{2-Layer MLP}(x) = \text{fc2}(\phi(\text{fc1}(x))) \quad (17)$$

Unlike CATS [24], which assesses importance based solely on activation magnitude, *AdaVLA* accounts for the scaling effect of the subsequent projection layer. We define the importance I_i of the i -th channel as the product of its activation magnitude and the corresponding projection weight norm:

$$I_i^{\text{gated}} = \|\phi(\text{Gate}(x)) \odot \text{Up}(x)\|_i \| \mathbf{w}_{\text{Down},i} \|_2 \quad (18)$$

$$I_i^{\text{2-Layer}} = \|\phi(\text{fc1}(x))\|_i \| \mathbf{w}_{\text{fc2},i} \|_2 \quad (19)$$

where $[\cdot]_i$ denotes the i -th channel's activation tensor across the sequence length, and $\mathbf{w}_i$ denotes the weight vector associated with the i -th intermediate channel. We sort the intermediate channels by importance and retain only the top fraction to ensure that critical features are preserved, thereby preventing performance degradation during the pruning process. While reordering could incur overhead, it is performed only during the initial forward pass or upon reaching the maximum number of allowed iterations, indicating a significant context shift.

V. EXPERIMENTS

A. Experimental Setup

Models: To validate the effectiveness of *AdaVLA*, we evaluated it on two representative state-of-the-art flow-matching-based VLAs: $\pi_{0.5}$ [11] and X-VLA [13] with a default step size of $s = 0.1$ (10 steps).

Simulation Benchmarks: We employed the LIBERO benchmark [14], which comprises the Spatial, Object, Goal, and Long task suites, to evaluate diverse semantic and spatial reasoning capabilities. We set the curvature threshold C_{th} to 0.15 for $\pi_{0.5}$ and 0.002 for X-VLA. These values were determined via a simple grid search, and our ablation studies demonstrate that performance is robust across a range of C_{th} values. For a new model or benchmark, users can readily identify a near-optimal C_{th} from a single profiling run by inspecting the resulting curvature distribution. All model checkpoints were obtained from the official LeRobot repository on Hugging Face [30], [31].

Hardware & Metrics: Following the evaluation protocol in [22], we evaluated each task over 50 rollouts. To avoid prohibitively long evaluation times on the edge device (up to 12 hours per full benchmark), the average success rate was measured on a server-grade NVIDIA A10 GPU. We additionally verified on a subset of tasks that the success rates on the edge-grade NVIDIA Jetson AGX Orin were comparable to those obtained on the A10. Latency was measured as the time required to generate a single action chunk on both the A10 GPU and the Jetson AGX Orin. To reflect real-world edge constraints, the total system energy consumption (CPU, GPU, and motherboard) was measured on the Jetson AGX Orin. Both latency and energy consumption were averaged over 10 rollouts per task.

B. Main Results on LIBERO Benchmark

Table II presents the quantitative results on the LIBERO [14] benchmark for $\pi_{0.5}$ [11] and X-VLA [13]. Following our deployment setting, we compared against the original models and training-free acceleration methods that are directly applicable to pretrained checkpoints without additional retraining, distillation, or architecture-specific re-design, including SparseVLM [7] and FastV [8].

On the Jetson AGX Orin, *AdaVLA* accelerates $\pi_{0.5}$ by $1.87\times$ (reducing latency from 958.54 ms to 513.35 ms) while slightly improving the average success rate to 99.45% and effectively reducing energy consumption by $1.51\times$ from

TABLE III: Ablation study of *AdaVLA* with $\pi_{0.5}$ on the LIBERO benchmark using Jetson AGX Orin.

Variant	C_{th}	Avg SR (%)	Latency (ms)	Energy (J)
Baseline (10-step)	-	98.65	958.54	19.83
Ours (+PR, +Adp)		99.45	513.35	13.17
Ours (+SVD, +Adp)	0.15	99.65	894.78	23.60
Ours (+Fxd, +Adp)		99.35	492.00	12.23
Ours (-Pruning, +Adp)	0.15	98.65	532.32	13.33
Ours (+PR, -Adp)		99.25	925.71	19.10
Ours (+PR, +Adp)	0.20	99.20	428.24	10.48
Ours (+PR, +Adp)	0.25	98.40	413.26	9.96
Baseline (1-step)	-	97.05	447.45	11.43
Baseline (5-step)	-	98.00	677.56	15.31
Baseline (Random-step)	-	98.65	694.13	15.49

19.83 J to 13.17 J. These efficiency gains are even more pronounced for smaller architectures like X-VLA, which are typically latency-bound due to kernel launch and memory access overheads. Previous methods [7], [8] target compute reduction solely within the VLM but fail to mitigate the underlying execution overheads. Consequently, they yield negligible speedups (e.g., FastV [8] reduces X-VLA latency by merely 2.3 ms on Jetson Orin). Conversely, by dynamically reducing flow matching steps, *AdaVLA* bypasses a substantial number of iterative forward passes, effectively alleviating the kernel launch and memory access overheads. For X-VLA, this advantage of *AdaVLA* translates to a $2.24\times$ latency reduction (from 387.52 ms to 173.09 ms) and a significant $3.06\times$ decrease in energy consumption (from 10.73 J to 3.51 J), while ensuring near-perfect success rates across all complex sub-tasks.

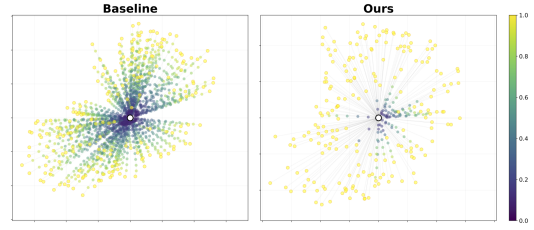
AdaVLA also maintains stable inference timing. On the Jetson AGX Orin, the standard deviations of per-step latency are 19.12 ms for $\pi_{0.5}$ and 21.06 ms for X-VLA, demonstrating the timing stability required for real-world deployment.

C. Ablation Study

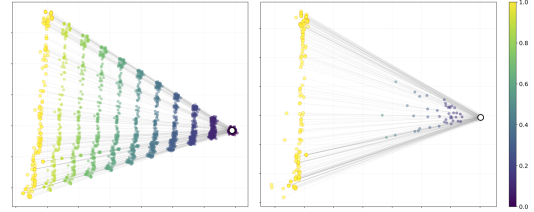
To evaluate the specific impact of each component, we conducted ablation studies on $\pi_{0.5}$ using the Jetson AGX Orin. The results are summarized in Table III.

Impact of Efficient Assessment: While SVD better identifies critical layers leading to a high success rate (99.65% SR), it offers limited latency improvements compared to our Participation Ratio-based pruning method (894.78 ms vs 513.35 ms). It also incurs an unacceptable Time-to-First-Action (TTFA) of 4939.32 ms and 23.60 J energy cost due to decomposition overheads. In contrast, the Participation Ratio (PR) approach obviates the need for explicit decomposition, utilizing computationally efficient trace and norm calculations instead. This yields a $5.63\times$ TTFA speedup (877.13 ms) and lowers energy usage to 13.17 J without meaningful performance degradation. Although PR adds a marginal 21 ms latency overhead compared to the fixed variant (+Fxd), it effectively preserves a 99.45% SR by accurately prioritizing representational density.

Necessity of Adaptive Inference and Pruning: Our adaptive components jointly optimize throughput and reliability.



(a) Trajectories of $\pi_{0.5}$ ($C_{th} = 0.15$)



(b) Trajectories of X-VLA ($C_{th} = 0.002$)

Fig. 3: Visualization of trajectories for $\pi_{0.5}$ and X-VLA during LIBERO-Long execution with default step size $s = 0.1$. The color gradient from dark to light represents the flow matching time variable $\tau \in [0, 1]$.

In particular, PR-based pruning safeguards model robustness. Removing the pruning component (-Pruning, +Adp) causes the success rate to revert to the 98.65% baseline, alongside slight increases in latency (532.32 ms) and energy (13.33 J). Simultaneously, adaptive inference is primarily responsible for latency control. Excluding this (i.e., +PR, -Adp) results in a substantial latency increase to 925.71 ms and an energy cost of 19.10 J, nearly matching baseline levels. Although the success rate remains at 99.25%, these observations suggest that adaptive inference is a key enabler for achieving real-time performance on edge devices, as its removal largely nullifies the latency gains of the proposed framework.

Sensitivity of Curvature Threshold: The curvature threshold C_{th} serves as a predictable control knob for the latency–success rate trade-off. Increasing C_{th} from 0.15 to 0.25 further reduces latency to 413.26 ms, at the cost of a marginal decline in the success rate to 98.40%. This stability suggests that the framework is not sensitive to threshold selection, providing flexible optimization across different hardware constraints without excessive dependence on the specific operating point. Depending on deployment requirements, users may prioritize lower latency or higher success rate; *AdaVLA* provides this flexibility through C_{th} , which can be selected via a lightweight grid search.

Comparison with Static Step Reduction: To determine if simply reducing flow matching steps can match our adaptive approach, we evaluated baseline variants with 1, 5, and random (1–10) flow matching steps. Results in Table III indicate that static reduction leads to decreased task reliability. For instance, a 1-step baseline achieves 447.45 ms latency, but its 97.05% success rate is lower than that of our adaptive method (99.45%). With 5-step or random-step configurations, the baseline success rates become comparable to that of our adaptive method, but their latencies increase to 677.56 ms and 694.13 ms, respectively. These observations suggest that



Fig. 4: Task description of real robot experiments

naive step reduction may limit the model’s capacity to navigate complex trajectories. Conversely, *AdaVLA* concentrates computational resources on critical segments, improving the latency–success rate trade-off.

D. Real-World Robot Experiments

To bridge the gap between simulation and physical deployment, we conducted real-world validation using the SO-ARM101 robot [15]. The entire inference process was executed on the NVIDIA Jetson AGX Orin, serving as a representative edge computing platform for mobile manipulators. We employed SmolVLA [12] as the base policy and fine-tuned it via imitation learning using 50 teleoperated expert demonstrations per task on a single A10 GPU. To evaluate the model’s versatility, we designed four distinct manipulation tasks requiring precise coordination:

- Task 1: Pick up the banana and place it in the basket.
- Task 2: Grab the socks and put them into the laundry bin.
- Task 3: Pick up the sausage and place it on the plate.
- Task 4: Pick up the cup and place it on the plate.

For the quantitative comparison, each task was evaluated over 30 independent trials. We measured the success rate and average latency, defined as the time required to generate a single action chunk on the Jetson AGX Orin. A uniform curvature threshold of 0.075 was applied across all tasks.

As summarized in Table IV, our framework achieves a substantial reduction in latency, averaging 488.49 ms compared to the 806.73 ms of the baseline, which translates to a $1.65\times$ speedup. Notably, this acceleration is achieved while also improving the overall average success rate from 87.50% to 90.00%, effectively offsetting the marginal performance degradation observed in specific scenarios (Tasks 3 and 4). This degradation primarily stems from coarser action trajectories induced by the larger integration steps, and can be readily mitigated by lightweight post-processing techniques such as action smoothing, which incur negligible

TABLE IV: Real-world experimental results on the SO-ARM101 robot using SmolVLA evaluated on the Jetson AGX Orin. Success rate is reported as (Successes / Trials).

Task	Success Rate		Latency (ms)	
	Baseline	Ours	Baseline	Ours
Task 1	25/30 (83.3%)	29/30 (96.7%)	803.49	496.79
Task 2	26/30 (86.7%)	28/30 (93.3%)	811.53	447.07
Task 3	29/30 (96.7%)	27/30 (90.0%)	805.31	476.00
Task 4	25/30 (83.3%)	24/30 (80.0%)	806.58	534.10
Average	87.50%	90.00%	806.73	488.49

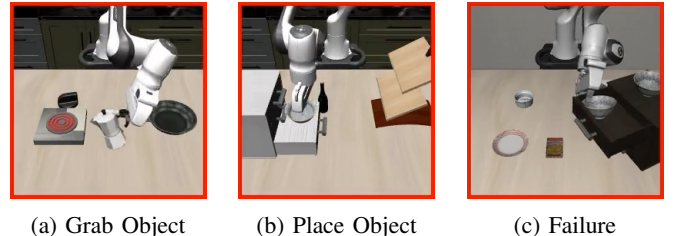


Fig. 5: Qualitative analysis of critical task segments where our framework adaptively allocates maximum inference steps

computational cost. These results underscore the efficacy of our adaptive approach, demonstrating that it can significantly accelerate inference speed without compromising overall operational reliability in real-world robotic deployments.

E. Analysis of Adaptive Inference

We visualized the actual trajectories generated by flow-matching-based VLAs to analyze the behavior of *AdaVLA*. Fig. 3 illustrates 250 randomly sampled trajectories recorded during the LIBERO-Long benchmark [14] for both $\pi_{0.5}$ [11] and X-VLA [13] under the same setup described in Sec. V-A.

In the baseline (left), both models consistently execute a high number of flow matching iterations, leading to a substantial number of forward passes in the Action Expert. In contrast, our method (right) generates clean actions with significantly fewer forward passes while adaptively allocating additional steps only when necessary.

Furthermore, we analyzed the specific task segments where these additional steps are triggered, particularly focusing on instances where the model reaches the maximum step limit. Our findings reveal that the model intensifies its computational effort during critical task moments, such as grasping (5a) and placing (5b) objects. Moreover, additional steps were dynamically invoked when the environment shifted unexpectedly—for instance, when an object fell during a pick-up attempt (5c). These results demonstrate that the curvature of the trajectories can serve as an intuitive proxy for perceived difficulty and action confidence, reducing reliance on complex, explicitly modeled difficulty estimators.

VI. CONCLUSION

We presented *AdaVLA*, a training-free, online adaptive framework for fast yet accurate flow-matching-based VLAs. Inspired by human cognition, our method autonomously regulates computational effort by utilizing flow-matching

trajectory curvature to adjust inference steps and MLP pruning ratios. On the Jetson AGX Orin, we achieved $1.87\times$ to $2.24\times$ speedup with negligible success rate degradation on the LIBERO benchmark and $1.65\times$ speedup in real-world experiments. By eliminating the need to access training data for fine-tuning or calibration, *AdaVLA* offers a highly practical solution for deploying flow-matching-based VLAs on resource-constrained robotic systems.

ACKNOWLEDGMENT

In accordance with the IROS guidelines on Generative AI usage, we explicitly state that we utilized Gemini 3 to refine the grammatical structure, improve readability, and generate specific illustrative elements for Fig. 1 and Fig. 2.

This work was partially supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (No. RS-2025-02263167).

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "OpenVLA: An open-source vision-language-action model," in *8th Annual Conference on Robot Learning*, 2024. [Online]. Available: <https://openreview.net/forum?id=ZMnD6QZAE6>
- [2] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.
- [3] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, T. Kreiman, Y. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine, "Octo: An open-source generalist robot policy," in *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.
- [4] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen *et al.*, "Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation," *IEEE Robotics and Automation Letters*, 2025.
- [5] Y. Yue, Y. Wang, B. Kang, Y. Han, S. Wang, S. Song, J. Feng, and G. Huang, "Deer-VLA: Dynamic inference of multimodal large language models for efficient robot execution," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [6] Y. Yang, Y. Wang, Z. Wen, L. Zhongwei, C. Zou, Z. Zhang, C. Wen, and L. Zhang, "EfficientVLA: Training-free acceleration and compression for vision-language-action models," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. [Online]. Available: <https://openreview.net/forum?id=SELYIDHZk2>
- [7] Y. Zhang, C.-K. Fan, J. Ma, W. Zheng, T. Huang, K. Cheng, D. A. Gudovskiy, T. Okuno, Y. Nakata, K. Keutzer, and S. Zhang, "SparseVLM: Visual token sparsification for efficient vision-language model inference," in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=80falPZ67S>
- [8] L. Chen, H. Zhao, T. Liu, S. Bai, J. Lin, C. Zhou, and B. Chang, "An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models," in *European Conference on Computer Vision*. Springer, 2024, pp. 19–35.
- [9] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=PqvMRDCJT9t>
- [10] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, "pi0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.
- [11] Physical Intelligence, K. Black, N. Brown, J. Darpanian, K. Dhaliwal, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, "pi0.5: a vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.
- [12] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti *et al.*, "Smolvla: A vision-language-action model for affordable and efficient robotics," *arXiv preprint arXiv:2506.01844*, 2025.
- [13] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, T. Wang, Y.-Q. Zhang, J. Liu, and X. Zhan, "X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=kt51kZH4aG>
- [14] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 44 776–44 791.
- [15] R. Knight, P. Kooijmans, T. Wolf, S. Alibert, M. Aractingi, D. Aubakirova, A. Zouitine, R. Martino, S. Palma, C. Pascal, and R. Cadene, "Standard open SO-100 & SO-101 arms," <https://github.com/TheRobotStudio/SO-ARM100>, 2024.
- [16] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu *et al.*, "Rt-1: Robotics transformer for real-world control at scale," in *Robotics: Science and Systems (RSS)*, 2023.
- [17] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Robotics: Science and Systems (RSS)*, 2023.
- [18] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang *et al.*, "Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation," *arXiv preprint arXiv:2411.19650*, 2024.
- [19] J. Wen, Y. Zhu, M. Zhu, Z. Tang, J. Li, Z. Zhou, X. Liu, C. Shen, Y. Peng, and F. Feng, "DiffusionVLA: Scaling robot foundation models via unified diffusion and autoregression," in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=VdwdU81Uzy>
- [20] J. Liu, M. Liu, Z. Wang, P. An, X. Li, K. Zhou, S. Yang, R. Zhang, Y. Guo, and S. Zhang, "Robomamba: Efficient vision-language-action model for robotic reasoning and manipulation," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [21] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *First conference on language modeling*, 2024.
- [22] M. J. Kim, C. Finn, and P. Liang, "Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success," in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [23] R. Zhang, M. Dong, Y. Zhang, L. Heng, X. Chi, G. Dai, L. Du, D. Wang, Y. Du, and S. Zhang, "Mole-vla: Dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation," *arXiv preprint arXiv:2503.20384*, 2025.
- [24] D. Lee, J. Lee, G. Zhang, M. Tiwari, and A. Mirhoseini, "CATS: Context-aware thresholding for sparsity in large language models," in *First Conference on Language Modeling*, 2024. [Online]. Available: <https://openreview.net/forum?id=v3w2a7EInO>
- [25] X. Wei, S. Moalla, R. Pascanu, and C. Gulcehre, "Building on efficient foundations: Effective training of llms with structured feedforward layers," *Advances in Neural Information Processing Systems*, vol. 37, pp. 4689–4717, 2024.
- [26] C. Runge, "Über die numerische auflösung von differentialgleichungen," *Mathematische Annalen*, vol. 46, no. 2, pp. 167–178, 1895.
- [27] W. Kutta, *Beitrag zur näherungsweise Integration totaler Differentialgleichungen*. Teubner, 1901.
- [28] O. Roy and M. Vetterli, "The effective rank: A measure of effective dimensionality," in *2007 15th European signal processing conference*. IEEE, 2007, pp. 606–610.
- [29] Q. Garrido, R. Balestriero, L. Najman, and Y. Lecun, "Rankme: Assessing the downstream performance of pretrained self-supervised representations by their rank," in *International conference on machine learning*. PMLR, 2023, pp. 10929–10974.
- [30] "LeRobot – $\pi_{0.5}$ libero finetuned checkpoint," https://huggingface.co/lerobot/pi05_libero_finetuned, 2025.
- [31] "LeRobot – x-vla libero checkpoint," <https://huggingface.co/lerobot/xvla-libero>, 2025.